Latest updates: https://dl.acm.org/doi/10.1109/ASE56229.2023.00038

RESEARCH-ARTICLE

DeepScaler: Holistic Autoscaling for Microservices Based on Spatiotemporal GNN with Adaptive Graph Learning

CHUNYANG MENG, Sun Yat-Sen University, Guangzhou, Guangdong, China

SHIJIE SONG, Sun Yat-Sen University, Guangzhou, Guangdong, China

HAOGANG TONG, Sun Yat-Sen University, Guangzhou, Guangdong, China

MAOLIN PAN, Sun Yat-Sen University, Guangzhou, Guangdong, China

YANG YU, Sun Yat-Sen University, Guangzhou, Guangdong, China

Open Access Support provided by:

Sun Yat-Sen University

PDF Download
ASE56229.2023.00038.pdf
07 April 2026
Total Citations: 2
Total Downloads: 115

Published: 26 September 2024

Citation in BibTeX format

ASE '23: 38th IEEE/ACM International Conference on Automated Software Engineering
November 11 - 15, 2023
Echternach, Luxembourg

Conference Sponsors:
SIGAI
SIGSOFT

ASE '23: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (November 2023)
https://doi.org/10.1109/ASE56229.2023.00038
ISBN: 9798350329964

2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE) | 979-8-3503-2996-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ASE56229.2023.00038

DeepScaler: Holistic Autoscaling for Microservices Based on Spatiotemporal GNN with Adaptive Graph Learning

Chunyang Meng†, Shijie Song†, Haogang Tong†, Maolin Pan‡ and Yang Yu‡,*
†*School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China*
‡*School of Software Engineering, Sun Yat-sen University, Guangzhou, China*
**Author to whom correspondence should be addressed*
Email: {mengchy3, songshj6, tonghg}@mail2.sysu.edu.cn, {panml, yuy}@mail.sysu.edu.cn

***Abstract*—Autoscaling functions provide the foundation for achieving elasticity in the modern cloud computing paradigm. It enables dynamic provisioning or de-provisioning resources for cloud software services and applications without human intervention to adapt to workload fluctuations. However, autoscaling microservice is challenging due to various factors. In particular, complex, time-varying service dependencies are difficult to quantify accurately and can lead to cascading effects when allocating resources. This paper presents *DeepScaler*, a deep learning-based holistic autoscaling approach for microservices that focus on coping with service dependencies to optimize service-level agreements (SLA) assurance and cost efficiency. *DeepScaler* employs (i) an expectation-maximization-based learning method to adaptively generate affinity matrices revealing service dependencies and (ii) an attention-based graph convolutional network to extract spatio-temporal features of microservices by aggregating neighbors' information of graph-structural data. Thus *DeepScaler* can capture more potential service dependencies and accurately estimate the resource requirements of all services under dynamic workloads. It allows *DeepScaler* to reconfigure the resources of the interacting services simultaneously in one resource provisioning operation, avoiding the cascading effect caused by service dependencies. Experimental results demonstrate that our method implements a more effective autoscaling mechanism for microservice that not only allocates resources accurately but also adapts to dependencies changes, significantly reducing SLA violations by an average of 41% at lower costs.**

***Index Terms*—Cloud Computing, Microservice, QoS, Resource Management, Holistic Autoscaling, Graph Convolution, Container**

I. Introduction

Cloud computing offers a well-consolidated paradigm for on-demand services provisioning on a pay-as-you-go basis [1]. It has gained more popularity in the last decade, as evidenced by the widespread use of cloud-based software services and applications [2]. One of its central attributes is elasticity, which enables flexible provisioning or de-provisioning computing resources according to customers' needs [3]. To efficiently employ the elasticity, acquiring and releasing cloud resources automatically and timely is vital, as human intervention is difficult or even impossible [4]. This mechanism of automatic and rapid scale outward and inward commensurate with demands under fluctuating workloads to guarantee quality of service (QoS) at minimal costs is called autoscaling [5].

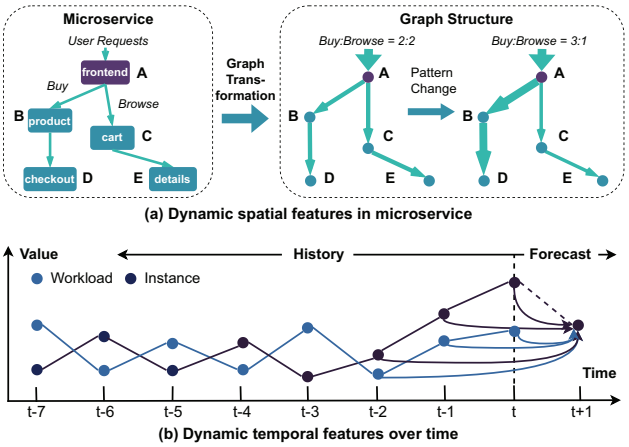

Fig. 1. Dynamic Spatial-temporal Features in Microservice

As cloud applications continue to grow in size and complexity, traditional monolithic applications face the problem of high development and maintenance costs [6]. The microservice architecture is proposed to address this problem, which splits single-component applications into multiple loosely coupled and self-contained microservice components [7]. Nonetheless, designing and implementing an efficient autoscaler for microservices is a challenging task due to various factors. Specifically, the primary challenges are as follows:

- **Complex dependencies.** In microservice applications, there are complex dependencies between internal services that can cause cascading effects when provisioning resources for interrelated services [8].
- **Dynamic environments.** Cloud-native microservices serve dynamic workloads with time-varying patterns and frequent software updates of microservices lead to changes of inter-dependencies [9].
- **QoS and cost trade-off.** There is a trade-off between

2643-1572/23/$31.00 ©2023 IEEE
DOI 10.1109/ASE56229.2023.00038

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

QoS and costs, since over-provisioning leads to increased costs, while under-provisioning violates service-level agreements (SLA). [10].

Research Gaps. Extensive related works have been proposed on autoscaling for microservices, including rule-based [11]–[14] and learning-based [15]–[17] methods. However, these works mainly confront two problems. First, most of them are insensitive to graph information that cannot effectively capture spatio-temporal characteristics of microservices (Fig. 1), usually scaling resources based on performance bottleneck services independently instead of holistically, resulting in distorted modeling and suboptimal resource allocation. Secondly, partial approaches consider service dependencies only in surface invocation relationships or rely on prior-knowledge-based handcraft dependency graphs. They are usually inadequate and lack adaptive extraction of other latent dependencies (e.g., causal relationships) hidden in the real data.

The *DeepScaler*. To fill these gaps and address the challenges, we present a deep learning-based holistic autoscaling method for microservice, namely *DeepScaler*. It aims to automatically re-provision resources of all scaling-needed services simultaneously according to precise resource estimation on dynamic workloads to optimize runtime QoS assurance and cost efficiency. *DeepScaler* consists of four procedures, including a performance monitor, a resource estimator, an adaptive learning module, and an online scheduler. *DeepScaler* first uses the performance monitor to collect and store tracing and telemetry data of microservices. Then, it uses the adaptive learning module to train the resource estimator and learn affinity matrixes that reveal service dependencies. Finally, it periodically uses the trained resource estimator, the learned affinity matrix, and the collected real-time performance indicators to predict future resource demands and allocate resources accordingly.

Contributions. To the best of our knowledge, this is the first work to provide holistic autoscaling for large-scale microservice applications by using spatio-temporal neural network with adaptive graph learning. Generally, our main contributions are:

- ***Resource Estimation*:** We present a deep learning-based resource estimation method for microservices. It utilizes attention based graph neural network that can capture the spatio-temporal features of microservices to estimate resource, making the estimation more accurate.
- ***Adaptive Learning*:** We propose an adaptive learning method based on expectation maximization (EM) and parametric graph learning for the above estimation model, which can capture latent service dependencies adaptively.
- ***Holistic Autoscaling*:** We present *DeepScaler*, holistic autoscaling for microservices, simultaneously allocating resources for interacting services in one scaling action, mitigating SLA violations while costing less.
- ***Implementation & Evaluation*:** We implement and evaluate *DeepScaler* based on an Istio-enabled Kubernetes container orchestration system in a public cloud environment. The experimental result shows that *DeepScaler* is a promising approach.

II. Background & Characterization

A. Microservice

Currently, cloud-based software services and applications have acquired popularity, encompassing all spheres of human activity. Many of these applications require interactivity and have strict performance requirements (high throughput and minimal tail latency) while also needing to ensure availability and manage frequent software updates [18], [19]. Large cloud providers such as Amazon, Twitter, Netflix, Apple, and eBay have adopted a significant design change to address the challenge of managing the constantly increasing size and complexity of their systems and meeting conflicting demands by moving away from traditional monolithic architecture and instead using graphs consisting of tens or hundreds of single-purpose, loosely-coupled microservices [20], [21]. Fig. 2 illustrates the difference between monolithic and microservices architectures.

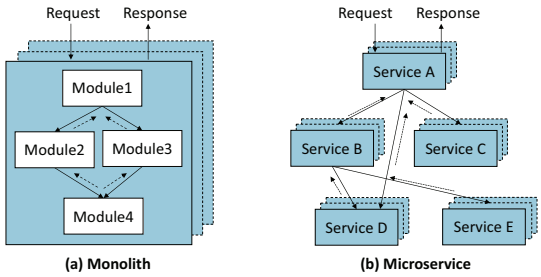

Fig. 2. Monolith and Microservice Architectures

Several compelling reasons substantiate the surging popularity of microservices. First, they foster composable software design, streamlining and accelerating development processes, with each microservice catering to a small subset of the application's functionality. Second, microservices facilitate programming language and framework heterogeneity, allowing each tier to develop in the most suitable language, with only a standard application programming interface (API), e.g., remote procedure calls (RPC) [22] or representational state transfer (RESTful) [23], required for inter-microservice communication. Finally, microservices simplify the resolution of correctness and performance issues, as bugs can isolate to specific tiers, unlike monoliths, where rectifying bugs necessitates troubleshooting the entire service. Despite the numerous benefits of microservices, they substantially challenge many assumptions on the design of current cloud systems. As a result, they pose both opportunities and challenges in terms of optimizing QoS and utilization.

B. Graph Neural Network

Graph Neural Network (GNN) is a type of neural networks that learn and infer information from graph-structured data [24]. It combines graph embedding techniques and neural network operators to learn latent representations for nodes in a

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

graph by taking into account the features of their neighboring node [25]. GNNs have gained significant attention in recent years due to its success in a variety of tasks such as node classification, link prediction, and social network analysis, among others.

In GNNs, input involves a graph represented by an adjacency matrix and a feature matrix. The adjacency matrix captures the relationships between nodes, while the feature matrix encapsulates the attributes or characteristics of each node. For example, in a social graph, nodes represent users with feature vectors denoting characteristics like age, location, and preferences, and links between users signify relationships like friendships. The output of a GNN model is termed the representation vector or embedding, a multi-dimensional vector encapsulating latent insights about nodes. For the social graph just mentioned, a user's embedding could be a numeric vector reflecting their attributes. By transforming the intricate semantics of raw data into well-structured numerical forms, these embeddings facilitate an array of downstream analytical tasks, including user identity classification [26], friendship prediction [27], and community detection [28].

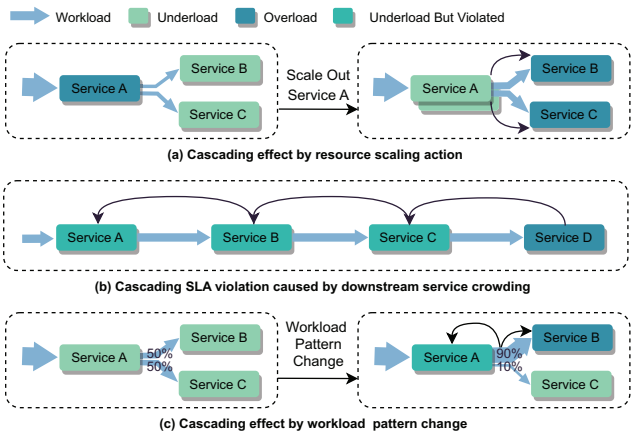

Fig. 3. Cascading Effects in Microservices

C. Key Insights

As mentioned earlier in Research Gaps of section I, although there have been some studies on microservice autoscaling, they lack more careful considerations on service dependencies, limiting the performance of their algorithms. To better understand the dependencies between microservices and study SLA violation characteristics, we have run extensive autoscaling experiments on widely used microservice benchmarks (i.e. Train-Ticket [29], Online-Boutique [30] and BookInfo [31]) in a public cloud environment. Our key insights are as follows.

Insight 1: Holistic Autoscaling is Necessary. Properly selecting the exemplary service for resource scaling in microservices is critical to mitigating SLA violations. Some algorithms [13], [32], [33] choose to scale the current performance bottleneck service, but our study shows that scaling only the bottleneck service may take more time to mitigate SLA violations. As shown in Fig. 3(a), consider that service A depends on its downstream services B and C. When resources are allocated to bottleneck service A with a large workload, A's ability to process requests increases, and the request rate at which A dispatched downstream by A increases, leading to the possibility that downstream services B and C may become new bottleneck services, resulting in an inability to mitigate SLA violations immediately.

> **Definition** *2.1: Holistic autoscaling* is a kind of autoscaling method that allows inter-related services to simultaneously re-provision during resource allocation to reduce the cascading impact caused by service dependencies.

Although the SLA violations can be completely mitigated by multiple scaling actions to eliminate the bottleneck services, it undoubtedly prolongs the SLA violation time. Our study indicates that, in microservice autoscaling, the optimal approach should consider service dependencies and interrelated microservices should be scaled simultaneously to avoid bottleneck shifting, thus mitigating SLA violations immediately.

Insight 2: Service dependencies must be discovered adaptively. Recent approaches have explored service dependencies of microservices and established corresponding holistic autoscaling methods to guarantee runtime QoS and reduce resource consumption. For example, Meng et al. [34] automatically captures potential service dependencies from global application-level metrics via deep reinforcement learning (DRL) to guide scaling decisions, Tong et al. [35] constructs balanced queuing networks based on expert-driven predefined dependency graphs that enable scaling all services simultaneously, and Yu et al. [36] automatically constructs service invocation relationships via online telemetry traces to determine which services need to be scaled.

> **Definition** *2.2: Latent service dependencies* belong to service dependencies, which describe the dependencies adaptively discovered from actual data rather than provided as ground truth knowledge. They may be some highly abstract dependencies that are difficult to understand.

However, our study shows that this is not sufficient by itself. The requirement is to capture the time-varying dependencies adaptively and as accurately as possible. Traditional neural networks are insensitive to graph structure information leading to modeling distortions. Expert-driven approaches face expensive model reconstruction engineering and difficulty adapting to dependency changes. The automatically constructed invocation graph based on telemetry traces only captures a type of communication-based dependencies between microservice instances, which cannot adequately describe other *latent service dependencies* (e.g., business correlations).

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

Insight 3: A single indicator, including request workload, resource utilization, and end-to-end latency, cannot identify the need for scaling. Many microservice autoscaling algorithms [37] determine the need for scaling based on a single performance indicator (e.g., workload, resource utilization, and end-to-end latency). However, our study shows that a single performance indicator is insufficient to identify the need for scaling. We explain below the reasons why using each of these three types of metrics alone is not sufficient.

First, it is not sufficient to use the size of resource utilization (e.g., CPU) to trigger scaling actions. For example, when a service has a high CPU utilization, it does not mean it needs to scale out. A software bug, such as an infinite loop, could cause it. Conversely, when the CPU utilization of a service is low, it does not mean that the service needs to scale in either because an unacceptable response latency may be caused by the limited availability of other underlying computational resources (e.g., memory) that need to be scaled up.

Second, as the end-to-end latency of a microservice depends not only on its own response time but also on the response time of its downstream services, it leads to confusion in deciding whether scaling is required based on the end-to-end latency. As shown in Fig. 3(b), when a downstream service is crowded, cascading SLA violations occur for the upstream services in its invocation chain.

Finally, since microservice workloads have dynamic load patterns, workloads will be dispatched to different downstream services in varying manner, making it impractical to determine the need for scaling based on the intensity of the workload. Consider service A and a large workload, where service A depends on downstream services B and C. As shown on the left of Fig. 3(c), the large workload is evenly distributed to different downstream services, resulting in service A providing an acceptable quality of service. Conversely, as shown on the right of Fig. 3(c), when the workload is congested on the same downstream service B, it may lead to an SLA violation in the response time of service A.

III. Problem Formulation And System Design

A. Problem Formulation

In this study, a microservice application is considered as a graph containing multiple interrelated services, denoted by $G = (V, A)$, where V is a finite set of nodes representing the services with $|V| = N$, and $A \in \mathbb{R}^{N \times N}$ represents the adjacency matrix of the service dependency. Each node in the microservice application G produces a feature vector of length C at each time step, where C is the number of online telemetry performance indicators collected with the same sampling frequency F.

The Problem of Microservice Autoscaling. Assume the c-th time series recorded on each node in the microservice application G is the instance replicas sequence, and $c \in (1, \ldots, C)$. We use $\mathbf{x}_t^{k,i} \in \mathbb{R}^C$ denotes the values of the k-th feature of node i at time t, and $\mathbf{x}_t^i \in \mathbb{R}^{N \times C}$ denotes the values of all the features of node i at time t. $\mathbf{X}_t = \left(\mathbf{x}_t^1, \mathbf{x}_t^2, \ldots, \mathbf{x}_t^N\right)^T \in \mathbb{R}^{N \times C}$ denotes the values of all the features of all the nodes at time t. $\mathcal{X}_t = (\mathbf{X}_{t-\tau+1}, \ldots, \mathbf{X}_{t-1}, \mathbf{X}_t)^T \in \mathbb{R}^{N \times C \times \tau}$ denotes the values of all the features of all the nodes on the past τ time slices at time t. At each discrete time step $t = 0, 1, 2 \ldots$,T, given $\mathcal{X}_t$, the goal is to find a sequence of instance replicas $\mathbf{Y}_t = \left(\mathbf{y}_t^1, \mathbf{y}_t^2, \ldots, \mathbf{y}_t^N\right)^T \in \mathbb{R}^N$ of all the nodes on the whole microservice application to minimize the SLA violation rate R with less resource cost $Cost$ in the total executing time, where $\mathbf{y}_t^i = x_{t+1}^{c,i} \in \mathbb{R}$ denotes the required instance replicas of node i at time $t+1$. The definitions of R and $Cost$ are shown in (1) and (2), respectively.

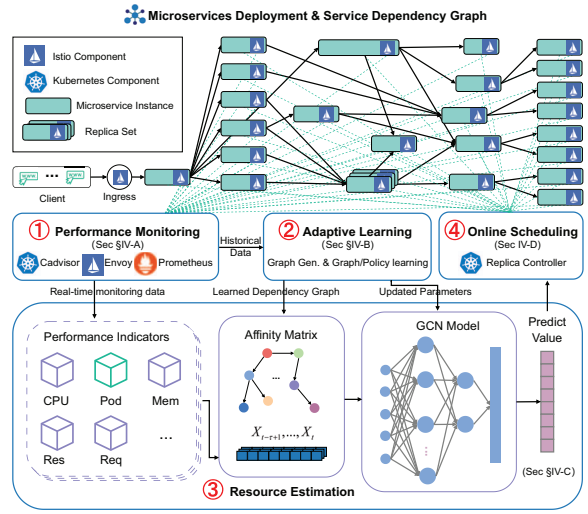

Fig. 4. System Overview

$$R = T_{violation}/T \tag{1}$$

$$Cost = \frac{F}{T^2} \sum_{t=0}^{T} \sum_{i=1}^{N} c_t^i \times y_t^i \tag{2}$$

where $T_{violation}$ is the cumulative time that the end-to-end response time of the microservice application violates the SLA requirements. c_t^i is the number of CPU logical cores configured for each instance of service i at time t.

B. System Design

To address the above problems, we propose a novel holistic autoscaling method based on spatio-temporal GNN and adaptive learning, namely *DeepScaler*, with the overview shown in Fig. 4 and pseudocode shown in Algorithm 1, which achieves runtime SLA guarantee and cost efficiency for microservices in dynamic environments. The system of *DeepScaler* is designed as a tight MAPE (Monitoring, Analysis, Planning, and Execution) control loop. We briefly introduce each phase here; the details can be found in §IV.

1) **Monitoring.** *DeepScaler* first needs to monitor some performance indicators to determine whether scaling operations are necessary and how they should be performed. It does so using the *Performance Monitor*, which is marked as ① and described in §IV-A. The

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

Algorithm 1: Pseudocode of *DeepScaler*

Initialize GNN-based resource estimator P with parameters θ and adjacency matrix A
Initialize telemetry dataset $\mathcal{D}$ and learning interval C
while not done **do**
  Observe current states $\mathbf{X}_t = \left(\mathbf{x}_t^1, \mathbf{x}_t^2, \ldots, \mathbf{x}_t^N\right)^T$ and store $\mathbf{X}_t$ in $\mathcal{D}$ according to §IV-A
  Every C time steps perform adptive learning (§IV-B):
$$A, P_\theta = AdaptLearning(A, P_\theta; \mathcal{D})$$
  Query last τ observations:
$$\mathcal{X}_t = (\mathbf{X}_{t-\tau+1}, \ldots, \mathbf{X}_{t-1}, \mathbf{X}_t)^T$$
  Predict future resource needs (§IV-C):
$$\mathbf{Y}_t = P_\theta(\mathcal{X}_t, A)$$
  Allocate resources according to $\mathbf{Y}_t$ (§IV-D)
end while

Performance Monitor collects tracing and telemetry data from every service instance in the microservice application and stores them in a centralized time-series database for processing.

2) **Analysis.** Then, the collected data are further processed in the analysis phase. *DeepScaler* periodically uses the *Adaptive Learning Module* (marked as 2 in Fig. 4 and described in §IV-B) with all the recently collected tracing and telemetry data to learn affinity matrices revealing the time-varying service dependencies and train the GNN-based *Resource Estimator*.
3) **Planning.** The planning phase estimates the resource demands of coming workloads for SLA assurance and resource efficiency optimization. By using the real-time data collected in 1 and the dependency graph learned in 2, and the fine-trained *Resource Estimator* (marked as 3 and described in §IV-C), *DeepScaler* makes online predictions of the instances required at the next time step for every service in the microservice.
4) **Execution.** Finally, in the execution phase, *DeepScaler* utilize the *Online Scheduler* (marked as 4 and described in §IV-D) to validate and execute resource scaling actions on the underlying Kubernetes cluster based on the service instances predicted in 3.

IV. The DeepScaler

A. Performance Monitoring

Performance indicators provide critical information about a system, allowing developers to (i) identify and address issues that may impact the user experience, (ii) estimate resources to support the system's current and future needs, and (iii) enable effective capacity planning. However, many cloud platforms do not offer adequate tools for monitoring applications at a highly detailed level, making it challenging to obtain accurate service-level performance indicators in real-world scenarios. While it is possible to achieve this through application-level instrumentation, developers must possess the necessary knowledge to expose these performance indicators.

In this study, microservices are deployed in a Kubernetes [38] cluster enabled with a service mesh (i.e., Istio). With the advantages of service mesh infrastructure, *DeepScaler* can easily manage microservices and collect performance indicators. Specifically, Istio mainly comprises a data plane and a control plane. The data plane is a composite entity that integrates a collection of intelligent proxies (i.e., Envoy [39]), which are strategically deployed as sidecars alongside microservices. These proxies adeptly mediate and control network communication between microservices. They also function as data collectors, collecting and reporting telemetry data pertinent to all mesh traffic. The control plane manages and configures the proxies to route traffic. The collected telemetry data is then stored in the Prometheus time-series database for querying. The data collected in our experiments are listed in Table I.

TABLE I
Collected Telemetry Data and Sources

Istio [40] **&** **Prometheus** [41]
istio_requests_total, istio_request_duration_milliseconds_sum, istio_request_duration_milliseconds_count
cAdvisor [42] **&** **Prometheus** [41]
container_spec_cpu_period, container_spec_cpu_quota, container_cpu_usage_seconds_total, container_memory_usage_bytes, container_spec_memory_limit_bytes

B. Adaptive Learning

As shown in Fig. 5, the EM-based adaptive learning is an iterative process, and each iteration contains three main steps: *Policy Learning*, *Graph Learning*, and *Graph Updating*.

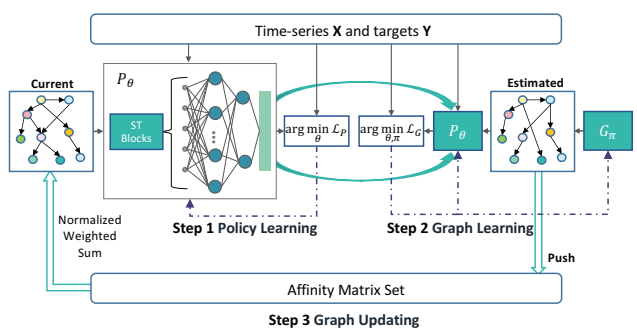

Fig. 5. Adaptive Learning Process

Policy Learning. *Policy learning* aims to optimise the parameters of the microservice resource estimator P (described in §IV-C) under the current affinity matrix of service dependency. The current affinity matrix A is initialized by a union operation on the affinity matrix set $\mathbb{A} = \{A_k | k = 1, 2, ..., N_r\}$. Each $A_k \in \mathbb{R}^{N \times N}$ denotes an affinity matrix of service dependencies, and $N_r = |\mathbb{A}|$ is the total number of graphs. The $\cup$ is defined as follows:

$$A^{(i,j)} = \frac{\sum_{k=1}^{N_r} A_k^{(i,j)}}{\sum_{k=1}^{N_r} \mathcal{I}[A_k^{(i,j)}]}; A_k \Leftarrow \widetilde{D}_k^{-\frac{1}{2}} \widetilde{A}_k \widetilde{D}_k^{-\frac{1}{2}} \tag{3}$$

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

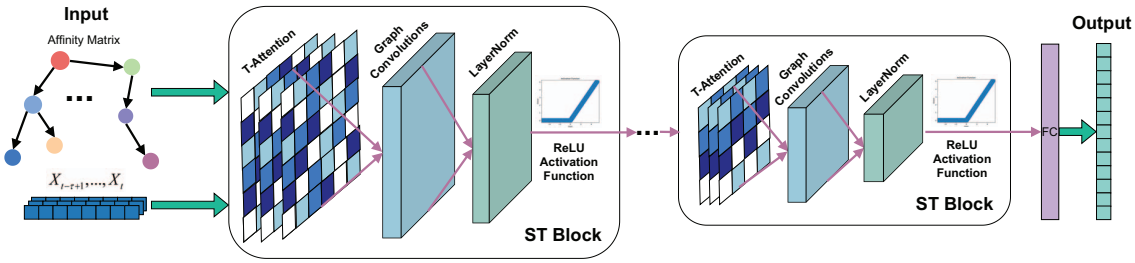

Fig. 6. Architecture of The Spatio-temporal GNN Model for Resource Estimation

where $\widetilde{D}_k^{ii} = \sum_j \widetilde{A}_k^{(ij)}$, $\widetilde{A}_k = A_k + I_N$, and $\mathcal{I}(x)$ represents the indicator function, which takes the value 1 when $x \neq 0$, and 0 otherwise.

The initial $\mathbb{A}$ contains an invocation relationship between microservices automatically constructed by a distributed tracing tool (i.e., Jaeger [43]). The loss function adopted in this step, represented as $\mathcal{L}_P$, is the L1 loss between the predicted sequence of instance replicas $\hat{\mathbf{Y}}$ and the ground truth $\mathbf{Y}$.

$$\mathcal{L}_P(\mathbf{Y}, \hat{\mathbf{Y}}) = |\mathbf{Y} - \hat{\mathbf{Y}}|_1 \tag{4}$$

Graph Learning. *Graph Learning* aims to optimize the parameters of the parameterized graph generator defined as follows:

$$\begin{aligned} G(A) &= Norm(S \odot A_1 + (1 - S) \odot A), \text{where} \\ A_1 &= \mathrm{ReLU}(M_1 M_2^T - M_2 M_1^T + \mathrm{Diag}(\Lambda)), S = g(h([A_1, A])) \end{aligned} \tag{5}$$

$$Norm(A) = D_2^{-\frac{1}{2}} A_2 D_2^{-\frac{1}{2}}; A_2 = \mathrm{ReLU}(D^{-\frac{1}{2}} A D^{-\frac{1}{2}} - \epsilon) \tag{6}$$

where g denotes a non-linear activation function (i.e., Sigmoid), h is one or more 1×1 convolutional layers, $\odot$ is the element-wise multiplication, D_2 is diagonal matrices and $D_2^{(i,i)} = \sum_{j=1}^{N} A_2^{(i,j)}$, and $\epsilon \in (0, 1)$ is a threshold used to filter out weak relationships.

The generated adjacent matrix is pushed into the affinity matrix set $\mathbb{A}$. Equations (5) and (6) ensure the overall sparsity of the generated matrix, while the number of related nodes for each node is not restricted. It should be noted that some nodes may be strongly correlated with other nodes while others are relatively isolated. Therefore, the *Graph Learning* can generate more effective relationships between the studied nodes.

The loss function $\mathcal{L}_G$ adopted in this step is as following:

$$\mathcal{L}_G(\mathbf{Y}, \hat{\mathbf{Y}}) = \mathcal{L}_P(\mathbf{Y}, \hat{\mathbf{Y}}) + \mathrm{ReLU}(\frac{C}{N^2} - \delta)/\delta \tag{7}$$

$$\Delta A = \mathrm{ReLU}\left[\mathcal{I}\left(A_{\text{new}}\right) - \mathcal{I}\left(A_{\text{old}}\right)\right]; C = \sum_{i=1}^{N} \sum_{j=1}^{N} \Delta A^{(i,j)} \tag{8}$$

The right half of $\mathcal{L}_G$ controls the proportion of the new edge to the maximum edges of the graph by introducing a hyper-parameter $\delta \in (0, 1)$. The left half of $\mathcal{L}_G$ is $\mathcal{L}_P$, meaning that stronger correlations with the microservice resource estimation are highlighted after a few iterations, while weaker correlations are gradually erased.

Graph Updating. Due to the insufficient precision and efficacy of the subgraphs within the affinity matrix set $\mathbb{A}$. We leverage prediction errors to weigh the significance of distinct subgraphs as illustrated by (9), thereby attaining an optimal estimate for matrix A:

$$A = Norm(\sum_{i=1}^{N_r} \mathrm{softmax}(\max_{1 \leq i \leq N_r} L_i - \vec{l}) A_i) \tag{9}$$

$$\vec{l} = (L_1, \ldots, L_{N_r})^T; L_k = \mathcal{L}_P[P_\theta(\mathcal{X}|A_k), \mathbf{Y}] \tag{10}$$

where $A_k \in \mathbb{A}$, and θ is the model parameter of P trained in the *Policy Learning* step.

Algorithm 2: Pseudocode of Adaptive Learning

Input: The input data $\mathcal{X}$, target data $\mathbf{Y}$, initial affinity matrix set $\mathbb{A}$ with capacity N_{max}, the GNN-based estimator P with parameters θ, the graph generator G with trainable parameters π,
Output: Trained P_θ, maximum likelihood estimation of A

```
Set A_old = ⋃_{A_i ∈ 𝔸} A_i
while not done do
    Train P_θ by minimize the loss via Adam:
        θ ← arg min_θ L_P(Y, P(X, A_old; θ))
    Generate new affinity matrix:
        A_new = G_π(A_old), and push A_new into 𝔸
    Fix θ, train G_π by minimize the loss via Adam:
        π ← arg min_π L_G(Y, P_θ(X, A_new; π))
    Compute prediction loss of all subgraphs in 𝔸 by (4)
    if |𝔸| > N_max then
        Remove subgraph with the max prediction loss
    end if
    Compute A according to (9), and set A_old = A
end while
return P_θ, A
```

C. Resource Estimation Using Spatio-temporal GNN

Accurately estimating resources and integrating with an autoscaler is crucial for the efficient implementation of autoscaling microservices, ensuring optimal operation without incurring unnecessary costs or risking performance issues.

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

By leveraging GNN, we accurately estimate resource needs in complex and dynamic environments, enabling efficient resource management and optimal performance with minimal overhead in autoscaling architecture. After the rigorous collection and meticulous pre-processing of the data, the analyzed data is fed into the model, leveraging a multifaceted blend of spatial-temporal blocks that are skillfully and iteratively assembled to capture a remarkably broad and diverse spectrum of dynamic spatial-temporal relationships, seamlessly incorporating various key aspects as depicted in Figure 6, which mainly includes T-Attention, Graph convolutions, LayerNorm.

T-Attention. T-Attention layer utilized an attention mechanism to handle the temporal dependencies among various time intervals in the workload conditions.

$$\begin{aligned} E &= V_e \cdot \sigma(((\mathbf{X}^{(r-1)})^T U_1) U_2 (U_3 \mathbf{X}^{(r-1)}) + b_e) \\ E'_{i,j} &= \exp(E_{\mathrm{i,j}}) / \sum_{j=1}^{T_{r-1}} \exp(E_{\mathrm{i,j}}) \end{aligned} \tag{11}$$

We defined a temporal correlation matrix E, which semantically represents the strength of dependencies between each time i and j. The calculation of E is based on the varying inputs, and the learnable parameters $V_e, b_e \in \mathbb{R}^{T_{r-1} \times T_{r-1}}, U_1 \in \mathbb{R}^N, U_2 \in \mathbb{R}^{C_{r-1} \times N}, U_3 \in \mathbb{R}^{C_{r-1}}$. C^{r-1} is the number of channels of the input data and T_{r-1} is the length of the temporal dimension in the r_{th} layer. The normalized temporal attention matrix E is then applied directly to the input data $\mathbf{X}$ to dynamically adjust the input by merging relevant information. Equation (12) represents the adjusted input data after considering the temporal dependencies.

$$\hat{\mathbf{X}}^{(r-1)} = (\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_{T_{r-1}}) E' \in \mathbb{R}^{N \times C_{r-1} \times T_{r-1}} \tag{12}$$

Graph Convolutions. Graph convolutions layer captures essential relationships between microservices for network representation. We predict pod replicas based on historical data and adjacency matrices by using the ChebConv technique. This approach efficiently approximates functions and captures local graph structure information using Chebyshev polynomials. We handle irregular graph structures and reduce computational complexity by leveraging the graph spectrum. The similarity between the generated relationships ensures stability and consistency in the learning process, enabling faster convergence and effective use of prior knowledge.

$$H^{(l)} = \mathrm{ChebConv}(\hat{A}, H^{(l-1)}; \Theta^{(l)}) \tag{13}$$

As equation (13) shown, in the context of ChebConv, the activation function utilized is $ReLU(\sigma)$. The normalized adjacent matrix is denoted as $\hat{A}$. H^l refers to the hidden features of the l_{th} layer, while $\Theta^{(l)}$ represents the learnable parameters corresponding to the same layer.

LayerNorm. LayerNorm improves convergence and reduces vulnerability to input magnitude in neural networks during training, with a mathematical formula as follows.

$$LayNorm(X) = \frac{X - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta \tag{14}$$

X is the input data vector after convolution, μ and σ are the mean and standard deviation of X along each feature dimension, respectively. ϵ is a very small constant, $\odot$ denotes element-wise multiplication, and γ and β are learnable parameter vectors used to scale and shift the normalized data. Subsequently, we employ $ReLU$ activation function to introduce non-linearity to the output of the previous layer and help prevent the vanishing gradient problem.

Ultimately, after feature extraction by stacked spatio-temporal blocks, the resulting tensor is fed into a fully connected (*FC*) layer. The input tensor is flattened into a one-dimensional vector and undergoes a linear transformation with a matrix of learnable weights and biases. This ensures the output is given due consideration for accurate and consistent forecasting.

D. Online Scheduling

We create Docker [44] images for each microservice in the application and save them in a private Docker hub; our images include all necessary dependencies, allowing microservices to be quickly deployed on any platform with a container engine. During deployment, the online scheduler uses the API of Kubernetes python client [45] to specify the number of resources the service requires at the container level based on the service instances predicted by the resource estimator.

Specifically, the Kubernetes master node receives the request for the creation instruction of a pod with specific compute resources and forwards it to the API server. The scheduler then determines the most suitable worker nodes (i.e., host servers) to run the pod and notifies the kubelet agent on the selected worker node to create the pod. Upon completion of pod creation, kubelet informs the API server that the pod is running. During runtime, the scheduler can modify the resource configuration online by submitting an update request to the master node. In addition, the scheduler sets a trust threshold for the prediction model. Whenever the number of prediction errors or SLA violations exceeds the threshold, the scheduler reduces trust in the model and becomes more conservative in recycling resources.

V. Experimental Evaluation

In this section, we evaluate *DeepScaler* to answer two questions:

- How effective is *DeepScaler*?
- How adaptive is *DeepScaler*?

A. Experimental Settings

System Setup. The experimental evaluation was conducted on a distributed cluster deployed in a public *Elastic Compute Service* (ECS) platform. The cluster contains eight virtual machines (VMs) running Ubuntu 18.04 LTS operating system with kernel version 4.15.0. Half of the VMs each have a 12-core 2.2 GHz CPU, 24 GB memory, and 100GB disk. Each of the other VMs has a 24-core 2.2 GHz CPU, 32 GB memory, and 500 GB disk. All the VMs are in the same local area network to reduce the network jitters. We used the Kubernetes

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

container orchestration system to manage the deployment of microservices on the cluster and Istio service mesh to take over network traffic and provide load balancing.

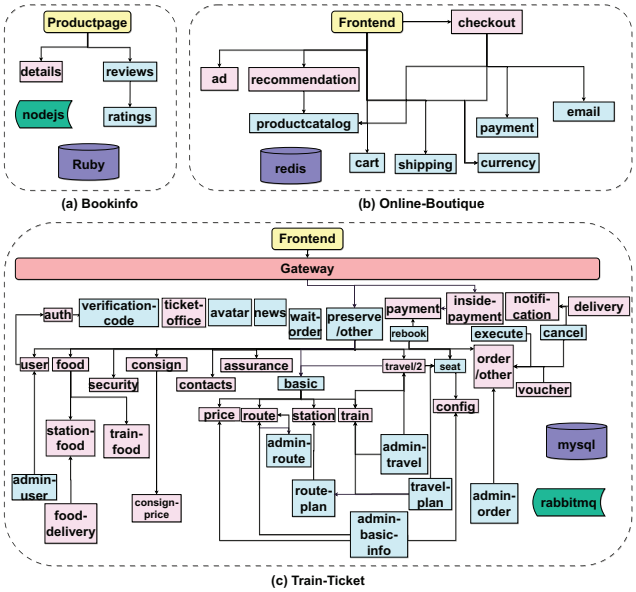

Fig. 7. Architectures of Used Benchmark Applications

Benchmark Microservices. The evaluation of *DeepScaler* utilizes three end-to-end interactive and responsive real-world microservice benchmarks. Fig. 7 shows the architectures of these applications, and a short description for each is as follows: (i) BookInfo [31], provided by Istio, is an online bookstore application that includes detailed information about a book, including its description, ISBN, number of pages, and reviews. The application comprises four microservices (Productpage, Reviews, Details, and Ratings), which have been incorporated to demonstrate various features of Istio. (ii) Online-Boutique [30] is a cloud-native microservices application used by Google to showcase the functionality of Kubernetes/GKE, Istio, and gRPC. This web-based e-commerce application allows users to browse a range of merchandise, add items to their cart, and complete their purchases. (iii) Train-Ticket [29] is a ticket booking application comprising 41 microservices, each responsible for a specific function, such as user authentication, ticket booking, payment processing, and notification, for a comprehensive evaluation in a multi-functional scenario. Various programming languages are used in benchmarks, such as C#, Java, Python, Go, Ruby, and Node.js.

Workload Generation. In order to replicate a live production environment, we employed Locust [46], an open-loop asynchronous workload generator, to drive the services. The generated workload intensity varied over time, emulating typical characteristics of microservice workloads, including slight increases, slight decreases, sharp increases, sharp decreases, and continuous fluctuations, as depicted in Fig. 8(a). All microservice benchmarks utilized this varied intensity of workloads, although each benchmark had distinct user request patterns. For instance, Online-Boutique's request pattern is presented in Fig. 8(b). To guarantee the stability of our experiments, we have deactivated all other user workloads on the cluster.

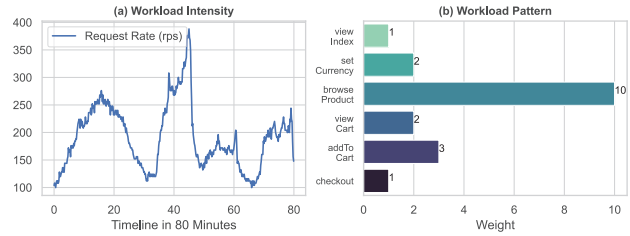

Fig. 8. Benchmark Workloads

Evaluation Metrics. To assess the precision of the model predictions, we employ mean absolute error (MAE), root mean square error (RMSE), and mean absolute percentage error (MAPE). We use the SLA violation rate, resource cost, and cumulative absolute errors (CAE) to appraise the scaling policy. The violation rate and resource cost are defined by (1) and (2). To evaluate whether *DeepScaler* extracted latent service dependencies, we employ the Jaccard Similarity to compare the relationships obtained from *DeepScaler* with those from origin-destination (OD) and correlation coefficient (CC) relationships. OD and CC relationships are defined as following:

$$A_{OD}^{(i,j)} = \begin{cases} N_{pas}(i,j) & \text{if } N_{pas}(i,j) > \frac{\max_j A_{OD}^{(i,j)}}{2} \\ 0 & \text{else} \end{cases} \tag{15}$$

$$A_{CC}^{(i,j)} = \begin{cases} \left| \frac{\mathrm{Cov}(\vec{s}_i, \vec{s}_j)}{\sqrt{\mathrm{var}(\vec{s}_i)\,\mathrm{var}(\vec{s}_j)}} \right| & \text{if } \left| \frac{\mathrm{Cov}(\vec{s}_i, \vec{s}_j)}{\sqrt{\mathrm{var}(\vec{s}_i)\,\mathrm{var}(\vec{s}_j)}} \right| > TC_i \\ 0 & \text{else} \end{cases} \tag{16}$$

where $N_{pas}(i,j)$ is the average number of requests from S_i to S_j, $\vec{s}_i$ stores historical data of node S_i and is replaced by its ranks.

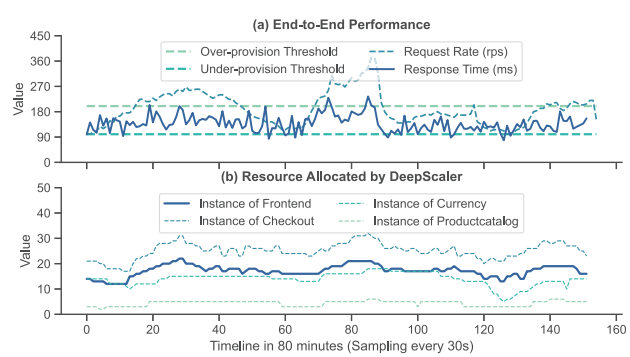

Fig. 9. A Running Example of *DeepScaler*

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

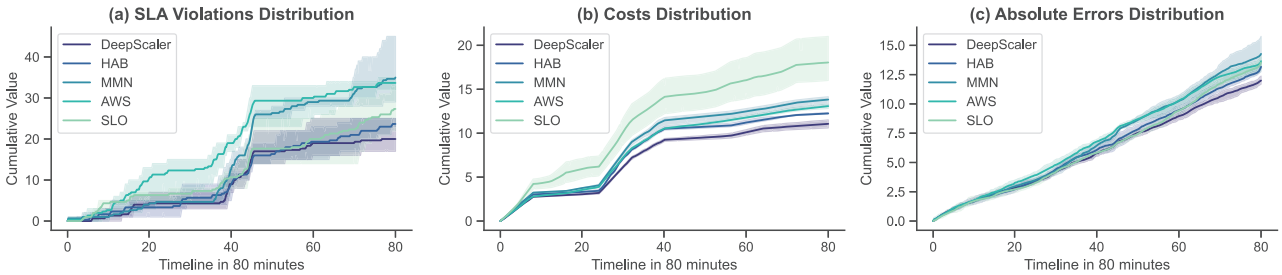

Fig. 10. Performance Comparison of Autoscaling Processes on SLA Violations, Costs, and Absolute Errors

B. Overall Evaluation Of the DeepScaler (RQ1)

Effectiveness. *DeepScaler* performs service instance prediction every 30 seconds based on real-time telemetry indicators and scales resources accordingly. Fig. 9 shows a running example of *DeepScaler* autoscaling the *Online Boutique* application under the benchmark workloads; we only plot the four most common services for clarity. From Fig. 9(a), we observe that *DeepScaler* controlling the vast majority of the end-to-end response time into a specific interval that neither violates SLA requirements nor over-provisioning. This fact implies that *DeepScaler* achieves an effective autoscaling mechanism that guarantees SLAs while reducing resource costs. We attribute this to *DeepScaler* correctly predicting resource needs and promptly scaling up and down the right services (as shown in Fig. 9(b)).

TABLE II
AUTOSCALING ASSESSMENT

Auto-scaler	Violation Rate *(percent)*	Cost *(core-hours)*	CAE *(secends)*
DeepScaler	**3.804±0.109**	**10.840±0.244**	**11.810±0.125**
HAB [35]	5.217±0.217	12.266±0.068	13.219±0.224
MMN [33]	8.043±1.739	13.673±0.466	14.260±1.500
AWS [47]	7.283±0.326	13.066±0.252	13.564±0.588
SLO [48]	5.978±1.196	18.510±2.497	13.581±0.409

Comparisons. To compare the performance between *DeepScaler* and other mainstream autoscalers, we implemented four baseline methods subjected to the same testbed as *DeepScaler* to manage all service resources. These four autoscaling methods are: (i) AWS [47] that emulates Amazon Auto-Scaling Services, (ii) MMN [33] based on an M/M/n/PS queuing model, (iii) HAB [35] based on a balanced queuing network, and (iv) SLO [48] based on SLO decomposition. AWS and SLO represent rule-based methods. AWS utilizes CPU utilization for rulemaking, while SLO relies on end-to-end response time. We compare these methods to assess the benefits of autoscaling using multiple indicators. HAB is a holistic autoscaling strategy considering service dependencies, while MMN is a non-holistic approach that independently models and scales. Both HAB and MMN are based on queueing theory. We contrast them to analyze the advantages of holistic autoscaling.

Table II shows a comparison of the quantitative metrics of the experimental results. From the table, we find that *DeepScaler* outperforms other methods in all metrics, with an average improvement of 41% in reducing SLA violations, 23% in reducing resource spending, and 13% in reducing cumulative errors. We argue that *DeepScaler* outperforms other approaches because it uses an elaborate GNN that can capture spatio-temporal features of graph-structural data to model microservice resource prediction and adaptively captures latent service dependencies using a graph learning method (§IV-B), making microservice resource prediction more accurate. In addition, Fig. 10 shows the cumulative distribution of these metrics over time. It can be seen that *DeepScaler*'s ability to reduce both SLA violations and resource costs consistently outperforms other methods over time and becomes increasingly evident. It means that in production environments where continuous operation is required, equipping *DeepScaler* not only provides better quality of service assurance but also delivers significant resource savings.

As observed in Table II and Fig. 10, the holistic autoscaling algorithm, i.e., *DeepScaler* and HAB, significantly outperforms other non-holistic autoscaling approaches. This fact implies that in microservice applications with complex service dependencies, the holistic autoscaling algorithms represent a promising solution for achieving QoS guarantees and optimal resource allocation policies. The reason is that the holistic autoscaling algorithm can scale both the bottleneck service and its associated services in one scaling action based on service dependencies, thus mitigating SLA violations as fast as possible. In contrast, the non-holistic approach takes more time to mitigate SLA violations.

It is also worth mentioning that at 30 to 50 minutes of the timeline axis in Fig. 10, the bursty workloads exhibit higher SLA violations, although all autoscaling methods have allocated more resources. That is because burst workloads change rapidly and have unpredictable emergence times and request volumes, challenging prediction-based proactive and rule-based reactive approaches. Therefore, studying further fine-grained handling of workload bursts is vital to improving the performance of all dynamic workload-oriented resource management algorithms.

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

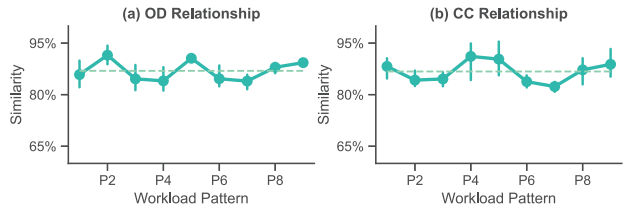

Fig. 11. Dependency Capture Capability on Different Workload Patterns

C. Adaptability Evaluation (RQ2)

Varying Different Workload Patterns. In practice, the workload intensity and patterns of microservices usually vary over time. §V-B has verified that *DeepScaler* can efficiently allocate resources under a specific pattern and varying intensity workload (as shown in Fig. 8). Here, we verify its adaptability to different workload patterns by tuning different weights of user request types. Fig. 11 shows the accuracy of *DeepScaler* in capturing service dependencies at different weights. From the figure, we observe that the accuracy in capturing OD and CC relationships always remains in an acceptable range (i.e., 89%±7%). It illustrates the adaptability of *DeepScaler* to workload pattern changes. Note that the initial affinity matrices of *DeepScaler* contain only invocation relationships. Therefore, the accurate capture of OD and CC relationships by *DeepScaler* validates its ability to adaptively capture latent service dependencies.

Varying Different Applications. To validate the adaptability of *DeepScaler* to different microservices and to evaluate the impact of potential service dependencies, we additionally implemented two GNN-based baseline algorithms: (i) a diffusion convolution recurrent neural network (DCRNN) [49] and (ii) a temporal graph convolutional network (T-GCN) [50] to perform resource prediction for the three different benchmark applications (§V-A). Fig. 12 displays the experimental results. From the figure, we find that although the prediction error of all algorithms gradually increases as the size of microservices increases, *DeepScaler* has the lowest error growth rate compared with other methods. That means *DeepScaler* has better adaptability to varying microservice application sizes and architectures. That is to say, *DeepScaler* is adaptive in managing the resources of large-scale microservices. The reason is that other GNNs only use predefined affinity matrices (i.e., invocation relationships), while *DeepScaler* is able to capture more potential service dependencies through an adaptive graph learning approach, thus enabling better adaptation to more complex microservice scenarios.

VI. Discussion

Alternate Design Choices. In our study, we employed a temporal attention-based graph convolutional network, a GNN, capturing spatiotemporal features of historical data to estimate the resource needs of microservices, and an EM-based graph learning module to discover service dependencies adaptively. While our proposed approach demonstrates effectiveness in achieving the desired outcomes, a rich landscape of spatiotemporal GNNs and graph learning methods could have been explored for similar purposes. For instance, alternative spatiotemporal GNNs, such as STFGNN [51], GraphSAGE [52], or GAT [53], could have been contemplated to apprehend diverse dimensions of the underlying spatiotemporal features within the data. Alternative graph learning techniques, encompassing the GRU of STGNN [54], STGSA [55], and GMAN [56], among others, could have been employed to unearth service dependencies in a varied manner. By delving into this rich landscape of possibilities, we can further enhance resource estimation's versatility, accuracy, and applicability for microservices autoscaling in various dynamic and evolving environments.

Overhead. Table III shows the overhead of *DeepScaler*. *Performance Monitoring* requires a resident time-series database (i.e., Prometheus) with an overhead that includes collecting and storing all performance indicators, which is about 64m core of CPU and 716 MB of memory. *Adaptive Learning* uses collected data to train the resource prediction model, which consumes 10.64 cores and takes 158.8 seconds. The retraining interval depends on whether changes in the microservice environment result in an unacceptable autoscaling performance. *DeepScaler* then performs *Resource Estimation* and corresponding *Online Scheduling* every 1 minute, consuming 0.107 cores of CPU and 232 MB of memory each time. In summary, *DeepScaler*'s overhead primarily comes from *Performance Monitoring*, which is essential for all autoscaling algorithms. Compared to the resources it can save and the benefits of QoS assurance, *DeepScaler*'s overhead is negligible.

TABLE III
The Overhead of DeepScaler

One-shot Operation	Overhead		
	CPU *(core)*	Mem *(MB)*	Time *(s)*
Performance Monitoring	64±23m	716±654	$+\infty$
Adaptive Learning	10640±300m	2108±212	158.8±2.1
Estimation & Scheduling	107±8m	232±34	0.5± 0.0

Limitation. First, *DeepScaler* is a data-driven approach limited by the quality and quantity of available historical data. Second, while *DeepScaler* can effectively mitigate SLA violations caused by workload fluctuations, it cannot detect and mitigate SLA violations caused by other system anomalies (e.g., network congestion, hardware failures, etc.). Finally, the current study focused on autoscaling microservices in a container-based cloud. How well *DeepScaler* would perform in other types of clouds needs to be clarified.

VII. Related Work

Autoscaling techniques have garnered substantial attention as a means to dynamically manage the allocation of computing resources in response to varying workload demands. This section comprehensively reviews autoscaling techniques,

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

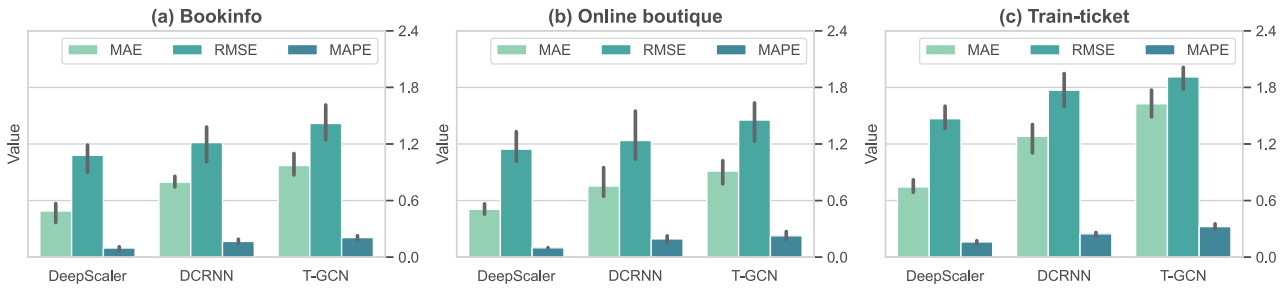

Fig. 12. Performance Comparison of Resource Estimations on Different Applications

categorized into distinct classes based on their underlying methodologies.

Rule-based autoscaling methods employ pre-defined thresholds and heuristics to trigger scaling actions [48], [57], [58]. Their simplicity and ease of implementation characterize these methods. Commonly used rules include CPU utilization thresholds, response time limits, and queue length triggers. Most cloud platforms, such as Google Cloud and Amazon Web Service, use this technique [47]. While straightforward, rule-based approaches may struggle to accommodate intricate workload dynamics, adapt to rapidly changing conditions, and optimize resource allocation under complex scenarios.

Model-based autoscaling methods employ formal mathematical models to establish relationships between workload characteristics and resource provisioning [59]–[61]. Queuing theory [33], [35], network models [62], and control theory [63] are frequently utilized in this category. These models provide a theoretical foundation for predicting system behavior and optimizing resource allocation. However, model-based approaches rely heavily on accurate system dynamics assumptions and may struggle to capture real-world intricacies and deviations.

Reinforcement Learning-based autoscaling paradigms empower systems to autonomously learn optimal scaling policies by iteratively exploring interactions with their environment [64]–[68]. For example, AutoMan et al. [17] propose a resource allocation method using multi-agent deep deterministic policy gradient in reinforcement learning to meet end-to-end tail latency SLOs for microservices. While inherently adaptive, these approaches often require substantial computational resources for training and necessitate careful consideration of exploration-exploitation trade-offs.

Machine Learning-based autoscaling techniques encompass a spectrum of data-driven methods, including supervised and unsupervised learning, to discern patterns and correlations within historical data [69]–[71]. For example, Sage et al. [72] used unsupervised ML models to avoid the cost of tracking tags, capture the impact of dependencies between microservices, and apply corrective measures to restore the QoS of cloud services. Li et al. [73] optimized the network overhead of microservice applications by utilizing the topology of microservices and clusters through a heuristic graph mapping algorithm. Compared to our work, these autoscaling methods superficially introduce service dependencies without dynamically adaptive modeling from a spatiotemporal perspective.

VIII. Conclusion and Future Work

In this paper, we present *DeepScaler*, a deep learning-based holistic autoscaling approach to manage resource allocation of containerized microservices under dynamic workloads to optimize SLA assurance and cost efficiency. With an elaborate attention-based GNN model and an adaptive graph learning method, *DeepScaler* can capture well microservices' spatio-temporal features and latent service dependencies to accurately estimate and proactively provision resources. Experiments validate the effectiveness and adaptability of *DeepScaler*. Compared with leading autoscaling approaches, *DeepScaler* obtains a more efficient resource allocation strategy and significantly reduces SLA violations at a lower cost. In the future, we intend to explore holistic autoscaling in extremely heterogeneous and geographically distributed computing resources to improve SLA guarantees for microservices under hybrid clouds. The source code of *DeepScaler* is available at https://github.com/SYSU-Workflow-Lab/DeepScaler.

Acknowledgements

We express our sincere gratitude to the reviewers for their insightful comments and constructive suggestions, which significantly contributed to the refinement of this paper. This research is supported by the NSFC-Guangdong Joint Fund Project (Grant No. U20A6003), the National Natural Science Foundation of China (NSFC) (Grant No. 61972427), and the Research Foundation of Science and Technology Plan Project in Guangdong Province (Grant No. 2020A0505100030).

References

[1] E. F. Coutinho, F. R. de Carvalho Sousa, P. A. L. Rego, D. G. Gomes, and J. N. de Souza, "Elasticity in cloud computing: a survey," *annals of telecommunications-annales des télécommunications*, vol. 70, pp. 289–309, 2015.

[2] T. Chen and R. Bahsoon, "Self-adaptive and online qos modeling for cloud-based software services," *IEEE Transactions on Software Engineering*, vol. 43, no. 5, pp. 453–475, 2017.

[3] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, "Elasticity in cloud computing: State of the art and research challenges," *IEEE Transactions on Services Computing*, vol. 11, no. 2, pp. 430–447, 2018.

[4] T. Chen, R. Bahsoon, and X. Yao, "A survey and taxonomy of self-aware and self-adaptive cloud autoscaling systems," *ACM Comput. Surv.*, vol. 51, no. 3, jun 2018. [Online]. Available: https://doi.org/10.1145/3190507

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

[5] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Comput. Surv.*, vol. 51, no. 4, jul 2018. [Online]. Available: https://doi.org/10.1145/3148149

[6] Z. Zhong, M. Xu, M. A. Rodriguez, C. Xu, and R. Buyya, "Machine learning-based orchestration of containers: A taxonomy and future directions," *ACM Comput. Surv.*, vol. 54, no. 10s, sep 2022. [Online]. Available: https://doi.org/10.1145/3510415

[7] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: yesterday, today, and tomorrow," *Present and ulterior software engineering*, pp. 195–216, 2017.

[8] Z. Yang, P. Nguyen, H. Jin, and K. Nahrstedt, "Miras: Model-based reinforcement learning for microservice resource allocation over scientific workflows," in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, 2019, pp. 122–132.

[9] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 3–18. [Online]. Available: https://doi.org/10.1145/3297858.3304013

[10] P. Singh, P. Gupta, K. Jyoti, and A. Nayyar, "Research on auto-scaling of web applications in cloud: survey, trends and future directions," *Scalable Computing: Practice and Experience*, vol. 20, no. 2, pp. 399–432, 2019.

[11] T. Daradkeh and A. Agarwal, "Modeling and optimizing micro-service based cloud elastic management system," *Simulation Modelling Practice and Theory*, vol. 123, p. 102713, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1569190X22001824

[12] A. Mirhosseini, S. Elnikety, and T. F. Wenisch, "Parslo: A gradient descent-based approach for near-optimal partial slo allotment in microservices," in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 442–457. [Online]. Available: https://doi.org/10.1145/3472883.3486985

[13] S. Horovitz and Y. Arian, "Efficient cloud auto-scaling with sla objective using q-learning," in *2018 IEEE 6th International Conference on Future Internet of Things and Cloud (FiCloud)*, 2018, pp. 85–92.

[14] B. Urgaonkar, P. Shenoy, A. Chandra, P. Goyal, and T. Wood, "Agile dynamic provisioning of multi-tier internet applications," *ACM Trans. Auton. Adapt. Syst.*, vol. 3, no. 1, mar 2008. [Online]. Available: https://doi.org/10.1145/1342171.1342172

[15] S. Zhang, T. Wu, M. Pan, C. Zhang, and Y. Yu, "A-sarsa: A predictive container auto-scaling algorithm based on reinforcement learning," in *2020 IEEE International Conference on Web Services (ICWS)*, 2020, pp. 489–497.

[16] A. Zafeiropoulos, E. Fotopoulou, N. Filinis, and S. Papavassiliou, "Reinforcement learning-assisted autoscaling mechanisms for serverless computing platforms," *Simulation Modelling Practice and Theory*, vol. 116, p. 102461, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1569190X21001507

[17] B. Cai, B. Wang, M. Yang, and Q. Guo, "Automan: Resource-efficient provisioning with tail latency guarantees for microservices," *Future Generation Computer Systems*, vol. 143, pp. 61–75, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X23000213

[18] C. Delimitrou and C. Kozyrakis, "Hcloud: Resource-efficient provisioning in shared cloud systems," *SIGARCH Comput. Archit. News*, vol. 44, no. 2, p. 473–488, mar 2016. [Online]. Available: https://doi.org/10.1145/2980024.2872365

[19] C. Delimitrou, D. Sanchez, and C. Kozyrakis, "Tarcil: Reconciling scheduling speed and quality in large shared clusters," in *Proceedings of the Sixth ACM Symposium on Cloud Computing*, ser. SoCC '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 97–110. [Online]. Available: https://doi.org/10.1145/2806777.2806779

[20] A. Cockcroft. (2016) Microservices workshop: Why, what, and how to get there. Accessed: Aug 10, 2023. [Online]. Available: https://www.slideshare.net/adriancockcroft/microservices-workshop-craft-conference

[21] ——. (2016) The evolution of microservices. Accessed: Aug 10, 2023. [Online]. Available: https://www.slideshare.net/adriancockcroft/evolution-of-microservices-craft-conference

[22] (2023) grpc. Accessed: Aug 10, 2023. [Online]. Available: https://grpc.io/

[23] L. Richardson and S. Ruby, *RESTful web services*. " O'Reilly Media, Inc.", 2008.

[24] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666651021000012

[25] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.

[26] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 249–270, 2022.

[27] A. Sankar, Y. Liu, J. Yu, and N. Shah, "Graph neural networks for friend ranking in large-scale social platforms," in *Proceedings of the Web Conference 2021*, ser. WWW '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2535–2546. [Online]. Available: https://doi.org/10.1145/3442381.3450120

[28] Z. Chen, X. Li, and J. Bruna, "Supervised community detection with line graph neural networks," *arXiv preprint arXiv:1705.08415*, 2017.

[29] (2022) Train Ticket: A Benchmark Microservice System. Accessed: Aug 10, 2023. [Online]. Available: https://github.com/FudanSELab/train-ticket

[30] (2023) Online Boutique: A microservice benchmark used by Google Cloud Platform. Accessed: Aug 10, 2023. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo

[31] (2023) Bookinfo: A microservice benchmark provided by Istio. Accessed: Aug 10, 2023. [Online]. Available: https://istio.io/docs/examples/bookinfo/

[32] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Automatic scaling for microservices with an online learning approach," in *2019 IEEE International Conference on Web Services (ICWS)*. IEEE, 2019, pp. 68–75.

[33] D. Jiang, G. Pierre, and C.-H. Chi, "Autonomous resource provisioning for multi-service web applications," in *Proceedings of the 19th International Conference on World Wide Web*, ser. WWW '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 471–480. [Online]. Available: https://doi.org/10.1145/1772690.1772739

[34] C. Meng, J. Tong, M. Pan, and Y. Yu, "Hra: An intelligent holistic resource autoscaling framework for multi-service applications," in *2022 IEEE International Conference on Web Services (ICWS)*, 2022, pp. 129–139.

[35] J. Tong, M. Wei, M. Pan, and Y. Yu, "A holistic auto-scaling algorithm for multi-service applications based on balanced queuing network," in *2021 IEEE International Conference on Web Services (ICWS)*, 2021, pp. 531–540.

[36] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Cost-effective scaling for microservice applications in the cloud with an online learning approach," *IEEE Transactions on Cloud Computing*, vol. 10, no. 2, pp. 1100–1116, 2022.

[37] C. Delimitrou and C. Kozyrakis, "Quality-of-service-aware scheduling in heterogeneous data centers with paragon," *IEEE Micro*, vol. 34, no. 3, pp. 17–30, 2014.

[38] (2023) Kubernetes: Production-grade container orchestration. Accessed: Aug 10, 2023. [Online]. Available: https://kubernetes.io/

[39] (2023) Envoy proxy. Accessed: Aug 10, 2023. [Online]. Available: https://www.envoyproxy.io/

[40] (2023) Istio: An open source service mesh that layers transparently onto existing distributed applications. Accessed: Aug 10, 2023. [Online]. Available: https://istio.io/

[41] (2023) The prometheus monitoring system and time series database. Accessed: Aug 10, 2023. [Online]. Available: https://github.com/prometheus/prometheus

[42] (2023) cadvisor: Container advisor. Accessed: Aug 10, 2023. [Online]. Available: https://github.com/google/cadvisor

[43] Cloud Native Computing Foundation. (2023) Jaeger: Distributed tracing system. Accessed: Aug 10, 2023. [Online]. Available: https://www.jaegertracing.io/

[44] Docker, Inc. (2023) Docker: Open platform for developing, shipping, and running applications. Accessed: Aug 10, 2023. [Online]. Available: https://www.docker.com/

[45] (2023) Python client for the kubernetes api. Accessed: Aug 10, 2023. [Online]. Available: https://github.com/kubernetes-client/python

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

[46] (2023) Locust performance testing tool. Accessed: Aug 10, 2023. [Online]. Available: https://locust.io/

[47] (2023) Amazon Auto Scaling Service. Accessed: Aug 10, 2023. [Online]. Available: http://aws.amazon.com/autoscaling/

[48] I. Gergin, B. Simmons, and M. Litoiu, "A decentralized autonomic architecture for performance control in the cloud," in *2014 IEEE International Conference on Cloud Engineering*, 2014, pp. 574–579.

[49] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," *arXiv preprint arXiv:1707.01926v3*, 2018.

[50] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, "T-gcn: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.

[51] M. Li and Z. Zhu, "Spatial-temporal fusion graph neural networks for traffic flow forecasting," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 5, pp. 4189–4196, May 2021. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/16542

[52] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7ebea9-Paper.pdf

[53] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio *et al.*, "Graph attention networks," *stat*, vol. 1050, no. 20, pp. 10–48 550, 2017.

[54] X. Wang, Y. Ma, Y. Wang, W. Jin, X. Wang, J. Tang, C. Jia, and J. Yu, "Traffic flow prediction via spatial temporal graph neural network," in *Proceedings of The Web Conference 2020*, ser. WWW '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1082–1092. [Online]. Available: https://doi.org/10.1145/3366423.3380186

[55] Z. Wei, H. Zhao, Z. Li, X. Bu, Y. Chen, X. Zhang, Y. Lv, and F.-Y. Wang, "Stgsa: A novel spatial-temporal graph synchronous aggregation model for traffic prediction," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 1, pp. 226–238, 2023.

[56] C. Zheng, X. Fan, C. Wang, and J. Qi, "Gman: A graph multi-attention network for traffic prediction," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, pp. 1234–1241, Apr. 2020. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/5477

[57] L. A. Vayghan, M. A. Saied, M. Toeroe, and F. Khendek, "A kubernetes controller for managing the availability of elastic microservice based stateful applications," *Journal of Systems and Software*, vol. 175, p. 110924, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121221000212

[58] S. N. Srirama, M. Adhikari, and S. Paul, "Application deployment using containers with auto-scaling for microservices in cloud environment," *Journal of Network and Computer Applications*, vol. 160, p. 102629, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S108480452030103X

[59] M. Yan, X. Liang, Z. Lu, J. Wu, and W. Zhang, "Hansel: Adaptive horizontal scaling of microservices using bi-lstm," *Applied Soft Computing*, vol. 105, p. 107216, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494621001393

[60] S. Wang, Z. Ding, and C. Jiang, "Elastic scheduling for microservice applications in clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 1, pp. 98–115, 2021.

[61] M. H. Fourati, S. Marzouk, and M. Jmaiel, "Epma: Elastic platform for microservices-based applications: Towards optimal resource elasticity," *Journal of Grid Computing*, vol. 20, no. 1, p. 6, 2022.

[62] M. Mekki, N. Toumi, and A. Ksentini, "Microservices configurations and the impact on the performance in cloud native environments," in *2022 IEEE 47th Conference on Local Computer Networks (LCN)*, 2022, pp. 239–244.

[63] B. Fodor, L. Toka, and B. Sonkoly, "Lsso: Long short-term scaling optimizer," in *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2022, pp. 52–58.

[64] M. Xu, C. Song, S. Ilager, S. S. Gill, J. Zhao, K. Ye, and C. Xu, "Coscal: Multifaceted scaling of microservices with reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 3995–4009, 2022.

[65] X. Chen, L. Yang, Z. Chen, G. Min, X. Zheng, and C. Rong, "Resource allocation with workload-time windows for cloud-based software services: A deep reinforcement learning approach," *IEEE Transactions on Cloud Computing*, vol. 11, no. 02, pp. 1871–1885, apr 2023.

[66] E. Karypiadis, A. Nikolakopoulos, A. Marinakis, V. Moulos, and T. Varvarigou, "Scal-e: An auto scaling agent for optimum big data load balancing in kubernetes environments," in *2022 International Conference on Computer, Information and Telecommunication Systems (CITS)*, 2022, pp. 1–5.

[67] F. Rossi, V. Cardellini, and F. L. Presti, "Hierarchical scaling of microservices in kubernetes," in *2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, 2020, pp. 28–37.

[68] A. A. Khaleq and I. Ra, "Intelligent autoscaling of microservices in the cloud for real-time applications," *IEEE Access*, vol. 9, pp. 35 464–35 476, 2021.

[69] Y. Song, C. Li, K. Zhuang, T. Ma, and T. Wo, "An automatic scaling system for online application with microservices architecture," in *2022 IEEE International Conference on Joint Cloud Computing (JCC)*, 2022, pp. 73–78.

[70] A. F. Baarzi and G. Kesidis, "Showar: Right-sizing and efficient scheduling of microservices," in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 427–441. [Online]. Available: https://doi.org/10.1145/3472883.3486999

[71] Y. Li, H. Zhang, W. Tian, and H. Ma, "Joint optimization of auto-scaling and adaptive service placement in edge computing," in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*, 2021, pp. 923–930.

[72] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: Ml-based and qos-aware resource management for cloud microservices," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 167–181. [Online]. Available: https://doi.org/10.1145/3445814.3446693

[73] X. Li, J. Zhou, X. Wei, D. Li, Z. Qian, J. Wu, X. Qin, and S. Lu, "Topology-aware scheduling framework for microservice applications in cloud," *IEEE Transactions on Parallel and Distributed Systems*, pp. 1–17, 2023.

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.